

estilos-de-arquitectura-de-backend

- Estilos de arquitectura de backend
 - 1. Contexto: de contener fallos a organizar el interior
 - 2. Capas, dominio y dependencias
 - 3. Domain-Driven Design: modelar lenguaje, reglas y límites
 - 4. Arquitectura Hexagonal (Ports & Adapters)
 - 5. Clean Architecture y la regla de dependencias
 - 6. CQRS y Event Sourcing: separar responsabilidades y decidir cómo persistir los cambios
 - 7. Arquitectura reactiva y programación reactiva
 - 8. Frameworks y bibliotecas como referencias
 - 9. Casos reales
 - 10. Diseño de referencia: el interior de Órdenes
 - 11. Trade-offs y criterios de decisión
 - 12. Puntos clave
 - 13. Referencias

Estilos de arquitectura de backend

1. Contexto: de contener fallos a organizar el interior

En las clases anteriores la plataforma de comercio pasó de un backend único a varias capacidades — Catálogo, Búsqueda, Órdenes, Pagos—, incorporó un borde de acceso y aprendió a contener fallos parciales. Esa distribución resuelve preguntas sobre despliegue, tráfico y coordinación. Dentro de cada servicio queda otra pregunta: ¿cómo se organizan las reglas de negocio y sus dependencias para que un cambio en una base de datos, framework o proveedor externo no obligue a reescribirlas?

Los alumnos ya conocen MVC y una organización por capas en Go. Esos estilos siguen siendo opciones válidas. Cuando una aplicación tiene pocas reglas y consultas sencillas, un Handler → Service → Repository con una base relacional puede ser la solución más clara. Cuando aparecen invariantes de negocio, modelos de lectura muy distintos, historial que debe reconstruirse o dependencias externas que cambian, conviene evaluar otras estructuras.

Esta clase compara decisiones que responden preguntas diferentes:

- **DDD** ayuda a comprender el negocio, expresar sus reglas y definir límites de modelo.
- **Hexagonal** y **Clean Architecture** organizan las dependencias para que el núcleo de la aplicación conserve sus reglas frente a detalles externos.
- **CQRS** separa las responsabilidades o modelos de lectura y escritura cuando sus necesidades divergen.

- **Event Sourcing** conserva hechos de negocio como registro canónico desde el cual reconstruir el estado.
- **La arquitectura reactiva** busca sistemas responsivos, resilientes, elásticos y orientados a mensajes; **la programación reactiva** compone operaciones asíncronas y flujos, y puede permitir I/O no bloqueante cuando todo el stack lo soporta.

No son piezas que haya que adoptar en conjunto. DDD puede usarse con una base relacional y CRUD; Hexagonal puede organizar un modelo sencillo sin DDD; CQRS puede separar lecturas y escrituras sin Event Sourcing; y un backend por capas puede ser suficiente si el problema no justifica más estructura. La decisión se toma según las reglas, los cambios probables, las cargas y los costos que el equipo puede sostener.

2. Capas, dominio y dependencias

2.1 MVC y arquitectura por capas

MVC organiza responsabilidades vinculadas con la interfaz: el **modelo** representa o coordina datos y comportamiento, la **vista** presenta información y el **controlador** interpreta acciones de entrada. En un backend HTTP, un Handler puede desempeñar parte del rol de entrada del controlador y una respuesta JSON puede ser una vista para otro cliente.

Una arquitectura por capas agrupa el código por tipo de responsabilidad. Un ejemplo conocido es:

```
HTTP Handler → Servicio de aplicación → Repositorio → Base de datos
```

La separación ayuda a localizar código y sustituir detalles con menos cambios. Sin embargo, los nombres de las carpetas no garantizan una frontera real. Si el servicio de aplicación importa tipos del ORM, las reglas quedan ligadas a la persistencia aunque exista una carpeta llamada `domain`. Si toda regla queda en el Handler, la capa de servicios no está protegiendo el dominio.

También conviene distinguir dos flujos:

- **Flujo de ejecución:** qué componente invoca a cuál durante una operación.
- **Dependencias del código:** qué componentes necesitan conocer los nombres y tipos de otros para compilar.

La primera flecha puede ir hacia afuera durante la ejecución aunque las dependencias del código apunten hacia el núcleo mediante interfaces. Hexagonal y Clean Architecture hacen explícita esta distinción.

2.2 Arquitectura, modelo de dominio y framework

Una arquitectura define elementos, responsabilidades, límites y relaciones relevantes para un sistema. DDD es un enfoque de análisis y modelado del dominio. CQRS y Event Sourcing son decisiones sobre el tratamiento de lecturas, escrituras y estado. La programación reactiva es un modelo para expresar operaciones asíncronas y flujos. Un framework brinda mecanismos de implementación.

Un framework HTTP como Gin puede recibir una solicitud y actuar como adaptador. No determina si la lógica se organiza con MVC, Hexagonal o Clean Architecture: esa frontera está en las dependencias y

responsabilidades que el equipo construye alrededor del framework.

3. Domain-Driven Design: modelar lenguaje, reglas y límites

3.1 El modelo debe representar el negocio

Domain-Driven Design (DDD) es un enfoque para desarrollar software cuando las reglas del negocio son suficientemente complejas como para que una representación técnica genérica produzca errores o cambios costosos. El equipo trabaja con expertos del dominio para construir un modelo que exprese esas reglas y use términos compartidos.

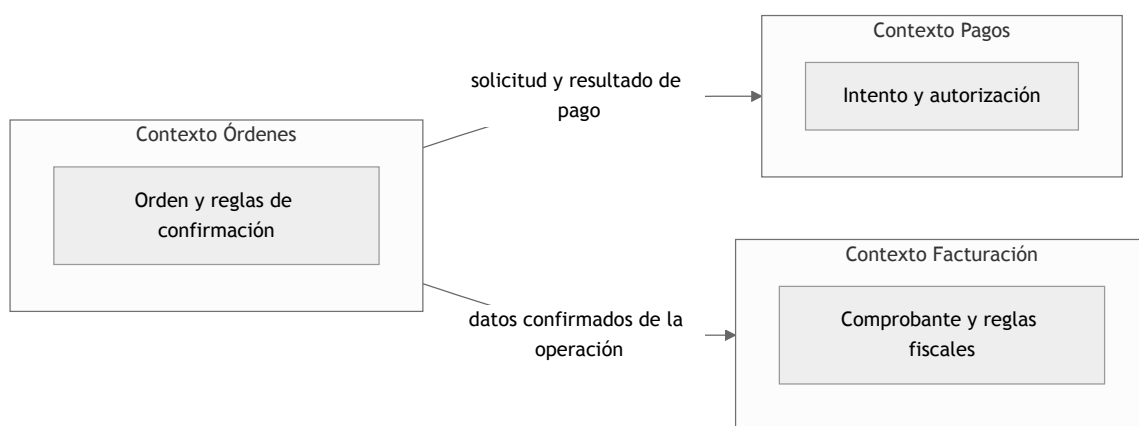
El **lenguaje ubicuo** es ese vocabulario compartido. Si el negocio distingue “orden confirmada” de “pago autorizado”, el código, las conversaciones y la documentación deberían conservar esa diferencia. Llamar a ambos estados `success` borra una distinción que puede ser importante para cancelar, facturar o notificar.

El modelo no es una copia literal del esquema de la base: una tabla describe almacenamiento, mientras que el modelo debe expresar conceptos y restricciones del negocio.

3.2 Subdominios y contextos delimitados

Un **subdominio** es una parte del problema de negocio con objetivos y reglas propios. Un **contexto delimitado** (*bounded context*) fija dónde aplica un modelo y su lenguaje. Dentro de ese límite, el término y las reglas tienen un significado coherente.

En la plataforma, Órdenes puede decidir si una compra cumple las reglas de confirmación; Pagos determina si una operación fue autorizada; Facturación conoce las reglas para emitir un comprobante. Los contextos se integran mediante contratos, pero cada uno conserva el modelo que necesita para su responsabilidad.



El diagrama muestra límites de modelo, no necesariamente servicios o bases separados. En un monolito, estos límites pueden expresarse mediante módulos. En una arquitectura distribuida pueden orientar servicios, pero el despliegue físico exige considerar también autonomía, operación, volumen, consistencia y ownership.

Un contexto debe traducir conceptos que recibe a su propio modelo. Si Pagos expone un `PaymentStatus` técnico, Órdenes puede mapearlo a sus estados de negocio en vez de depender de la enumeración y las

reglas internas del proveedor. Esta traducción evita que un vocabulario externo se filtre sin control.

3.3 Patrones tácticos para expresar reglas

Los patrones tácticos ayudan a modelar conceptos dentro de un contexto delimitado. No se agregan todos a cada objeto ni se mapean mecánicamente a tablas.

- **Entidad:** objeto cuya identidad persiste aunque cambien algunos atributos. Una Orden sigue siendo la misma orden aunque cambie de `Pendiente` a `Confirmada`.
- **Objeto de valor:** concepto definido por sus valores y no por identidad propia. Un `Monto` puede combinar cantidad y moneda; dos montos con iguales valores representan el mismo concepto. Suele ser inmutable para que el cambio se exprese creando un valor nuevo.
- **Agregado:** límite de consistencia que agrupa el estado que debe proteger una regla en una operación de negocio. La **raíz del agregado** es la entrada para aplicar cambios; las entidades internas no se modifican desde cualquier lugar.
- **Invariante:** regla que debe cumplirse al aceptar una transición. Por ejemplo, una orden cancelada no puede volver a estado confirmada mediante una edición arbitraria.
- **Servicio de dominio:** operación con reglas propias del dominio que no pertenece naturalmente a una entidad u objeto de valor particular.
- **Servicio de aplicación:** coordina un caso de uso —cargar un agregado, invocar su comportamiento, guardar el resultado— sin convertirse en el lugar donde se esconden las reglas del dominio.
- **Repositorio:** abstracción para recuperar y guardar agregados completos, no entidades sueltas. El repositorio se define para la raíz del agregado; una entidad interna —por ejemplo, una línea de una orden— se recupera y persiste a través de esa raíz, no mediante un repositorio independiente. El contrato usado por la lógica de aplicación no necesita exponer SQL, un ORM ni la forma física del almacenamiento.
- **Evento de dominio:** hecho significativo para el modelo, por ejemplo `OrdenConfirmada`. Un registro de base de datos o un cambio de campo no es automáticamente un evento de dominio.

Un agregado no equivale a una tabla ni a un conjunto grande de objetos relacionados. Debe ser lo bastante acotado para que sus reglas se mantengan dentro de una operación consistente. Entre agregados, el sistema puede coordinar procesos mediante sus identificadores y contratos, en lugar de mantener referencias a todos sus objetos internos.

Conviene fijar la diferencia entre tres límites que se suelen confundir: el **agregado** es un límite de consistencia transaccional, el **contexto delimitado** es un límite de lenguaje y modelo, y el **microservicio** es un límite de despliegue y ejecución — los tres pueden coincidir en un diseño, pero ninguno implica los otros dos. Por eso una operación de negocio cambia normalmente un solo agregado por transacción, con consistencia inmediata; si una regla necesita reflejarse en otro agregado —por ejemplo, ajustar un cupo disponible tras confirmar una orden—, esa coordinación suele resolverse con eventos de dominio y consistencia eventual entre agregados, en lugar de ampliar la transacción para que abarque a los dos.

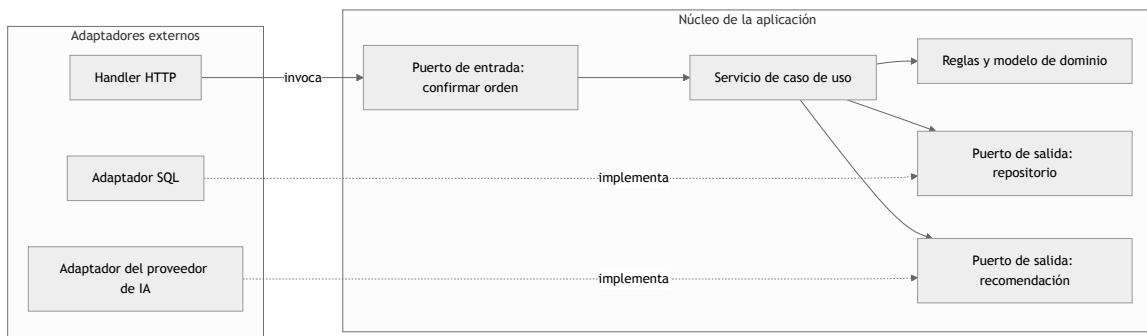
DDD aporta un lenguaje y herramientas para reconocer qué reglas y límites importan. Hexagonal o Clean Architecture pueden proteger esas reglas, con o sin un modelo DDD explícito. Del mismo modo, identificar un contexto no obliga a desplegarlo como microservicio.

4. Arquitectura Hexagonal (Ports & Adapters)

4.1 El núcleo declara sus necesidades

La Arquitectura Hexagonal, también llamada **Ports & Adapters**, protege el núcleo de la aplicación de los detalles que entran o salen. El núcleo contiene casos de uso y reglas de negocio. Los puertos expresan cómo se interactúa con él y qué capacidades externas necesita.

- Un **puerto de entrada** (*primary* o *driving port*) describe una capacidad que el sistema ofrece: confirmar una orden, cancelar una reserva o consultar una recomendación.
- Un **puerto de salida** (*secondary* o *driven port*) describe una necesidad del núcleo: guardar una orden, consultar disponibilidad o pedir una inferencia.
- Un **adaptador** traduce un mecanismo externo al contrato del puerto: un Handler HTTP puede ser un adaptador de entrada; un cliente SQL o un SDK de IA pueden ser adaptadores de salida.



Las flechas sólidas muestran invocaciones durante el caso de uso; las punteadas muestran que un adaptador implementa el contrato del puerto. Un puerto define una conversación o API con el núcleo; en Go suele expresarse mediante una interfaz cuando eso ayuda a fijar el contrato. El puerto de entrada describe una capacidad que ofrece el núcleo y el adaptador la invoca. El puerto de salida expresa una capacidad que necesita el caso de uso y el adaptador la implementa. La ubicación concreta de una interfaz puede variar; lo importante es que el núcleo no dependa de los tipos del adaptador externo.

4.2 Dirección de dependencias y sustitución

Supongamos que el caso de uso necesita generar una recomendación. Si importa directamente el SDK de un proveedor, el vocabulario, los errores y los formatos de ese proveedor penetran en las reglas internas. Para cambiarlo, el equipo debe modificar lógica de negocio y pruebas junto con la integración.

Con un puerto, el núcleo expresa la operación que necesita con términos propios. Un adaptador convierte esa solicitud al formato del proveedor A; otro la convierte al proveedor B. La política de negocio decide si utiliza la recomendación, la muestra como opcional o sigue sin ella. Cambiar el proveedor reemplaza o configura un adaptador, siempre que ambos satisfagan el contrato útil para el dominio.

Esta relación aplica el principio de inversión de dependencias: los componentes centrales no deben depender de detalles externos; los detalles se conectan a contratos estables definidos hacia adentro. El *dependency*

injection puede ensamblar los objetos en tiempo de ejecución, pero el contenedor de inyección por sí solo no garantiza que el núcleo esté aislado.

Una interfaz merece existir cuando delimita una variación relevante o permite sustituir una dependencia externa. Crear una interfaz para cada función, aunque nadie vaya a necesitar otro adaptador ni una prueba aislada, agrega indirection y mantenimiento sin mejorar la decisión.

4.3 Pruebas y costos

Los casos de uso pueden probarse con adaptadores falsos para verificar reglas sin iniciar una base de datos o llamar a un proveedor. Los adaptadores requieren pruebas que comprueben traducción de datos, configuración y comportamiento ante respuestas inválidas. También se necesitan pruebas de integración que comprueben que las piezas reales están conectadas correctamente.

Hexagonal facilita sustituir entradas y salidas y aislar reglas. A cambio, introduce contratos, conversiones y más piezas. Si la lógica del caso de uso termina repartida entre el Handler, el adaptador y el repositorio, la separación nominal no produjo un núcleo más claro.

4.4 Correspondencia con el proyecto de DSW

El esqueleto Handler → Service → Repository de DSW es un punto de partida familiar, pero la secuencia de capas no lo convierte por sí sola en Hexagonal. Para acercarlo a ese estilo, hay que hacer explícitos los contratos que necesita el núcleo y dirigir las dependencias hacia ellos, no hacia los detalles de infraestructura.

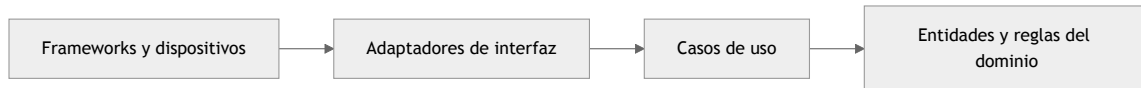
Concepto en DSW (Go + Gin + GORM)	Rol en Hexagonal
Handler de Gin	Adaptador de entrada
Interfaz que declara el caso de uso, por ejemplo <code>ConfirmarOrden</code>	Puerto de entrada
Interfaz que declara el acceso a datos, por ejemplo <code>OrderRepository</code>	Puerto de salida
Implementación de esa interfaz con GORM sobre MySQL	Adaptador de salida
Struct de dominio sin tags de <code>gorm</code> ni <code>json</code>	Entidad o agregado
Struct con tags <code>gorm:"..."</code> o <code>json:"..."</code>	Modelo de la capa externa — no debería cruzar hacia el núcleo

El cambio principal no es agregar una carpeta `domain`: es que el núcleo dependa de contratos útiles para sus casos de uso y que los adaptadores de Gin y GORM se conecten a esos contratos, en vez de filtrar tipos concretos de esos frameworks hacia las reglas de negocio.

5. Clean Architecture y la regla de dependencias

5.1 Anillos y regla de dependencias

Clean Architecture agrupa el sistema en anillos conceptuales que van desde las reglas del dominio hacia los mecanismos externos:



Las flechas representan dependencias del código hacia adentro. Las capas exteriores pueden conocer las interiores; las reglas interiores no importan frameworks, controladores HTTP, tipos de ORM ni SDKs. El flujo de ejecución puede cruzar estas fronteras en sentido inverso mediante interfaces, parámetros y objetos definidos por las capas internas.

Los grupos conceptuales suelen describirse así:

1. **Entidades y reglas de negocio:** invariantes que tienen significado duradero para el dominio.
2. **Casos de uso:** operaciones que coordinan reglas para cumplir objetivos concretos, como confirmar una orden.
3. **Adaptadores de interfaz:** convierten datos de transportes y almacenamientos a los formatos usados por las capas interiores.
4. **Frameworks y dispositivos:** detalles externos como HTTP, bases de datos, bibliotecas, sistemas operativos y proveedores.

5.2 Entidades y casos de uso: dos niveles de reglas de negocio

Las **entidades** expresan reglas de negocio de la empresa: invariantes que seguirían siendo válidas aunque este sistema en particular no existiera —por ejemplo, que una orden confirmada no puede facturarse por un monto distinto al pagado. En un diseño DDD, esas reglas pueden expresarse mediante entidades, agregados, objetos de valor e invariantes.

Los **casos de uso** expresan reglas de negocio de la aplicación: cómo esta aplicación concreta coordina esas entidades para cumplir un objetivo —qué pasos sigue `ConfirmarOrden`, qué valida antes de invocar al agregado, qué hace con el resultado—. Dos sistemas pueden compartir las mismas entidades y tener casos de uso distintos si automatizan el trabajo de forma diferente.

Esta distinción importa para decidir dónde vive una regla: si cambiaría en cualquier sistema que gestionara órdenes, es de la entidad; si depende de cómo esta aplicación decidió resolver el flujo, es del caso de uso.

5.3 Cruzar límites sin acoplar

El flujo de una solicitud cruza los anillos hacia adentro, pero su resultado necesita volver hacia afuera sin que un caso de uso importe tipos de la capa externa. La solución es la misma que usa Hexagonal con sus puertos de salida: el caso de uso define una interfaz de salida —un *output boundary*— con los datos que necesita

entregar, y una capa externa (un *presenter* o un adaptador) la implementa para transformarlos al formato que necesite el cliente (JSON, una vista, otro formato).

Así, el caso de uso nunca conoce el formato de la respuesta HTTP ni las anotaciones del serializador: solo conoce su propia interfaz de salida, definida hacia adentro. Es la misma inversión de dependencias que Hexagonal describe con puertos y adaptadores, aplicada al cruce de cada límite entre anillos.

5.4 Relación con Hexagonal y con DDD

Clean Architecture y Hexagonal comparten la intención de aislar el núcleo. Hexagonal destaca los puertos y adaptadores que conectan con el exterior; Clean Architecture representa anillos y una regla de dependencia. Los diagramas ayudan a pensar límites, pero no exigen una estructura fija de carpetas.

La aplicación de DDD puede aportar modelos e invariantes para proteger en los anillos interiores. Sin embargo, Clean Architecture puede organizar casos de uso con un modelo sencillo, y DDD puede implementarse con otras estructuras de dependencias. Son decisiones relacionadas que resuelven problemas distintos.

6. CQRS y Event Sourcing: separar responsabilidades y decidir cómo persistir los cambios

6.1 Comandos, consultas y modelos

Command Query Responsibility Segregation (CQRS) separa las operaciones de escritura —**comandos**— de las operaciones de lectura —**consultas**—. Un comando representa una intención de negocio y puede cambiar el estado. Una consulta devuelve la información solicitada sin modificarlo.

Operación	Ejemplo	Responsabilidad
Comando	<code>ConfirmarOrden</code>	Validar las reglas, proteger invariantes y aceptar o rechazar el cambio.
Consulta	<code>ConsultarHistorialDeOrden</code>	Devolver una vista con los datos que necesita la pantalla o consumidor.

Command-Query Separation (CQS) puede describir la separación en una función u objeto: una operación consulta o cambia. CQRS lleva una separación similar a los modelos y responsabilidades de una aplicación. Dividir funciones en `CommandHandler` y `QueryHandler` no significa que sea necesario tener dos bases de datos.

La adopción puede crecer por etapas:

1. Separar el código que valida cambios del código que prepara consultas, manteniendo un modelo y una base compartidos.

2. Definir modelos distintos: el modelo de escritura representa invariantes y transiciones; el de lectura devuelve proyecciones ajustadas a cada vista.
3. Separar almacenes físicos solo si los requisitos de escalado, tecnología, seguridad o volumen lo justifican.

Por ejemplo, `ConfirmarOrden` puede cargar la Orden, comprobar que admite la transición y persistirla. Una consulta del historial puede devolver una proyección con fecha, estado y resumen de pago, sin cargar el agregado completo ni incorporar su lógica de validación.

El mismo modelo de escritura puede alimentar más de un modelo de lectura. Una consulta de historial para el cliente no necesita la misma forma que un panel interno de seguimiento de órdenes: cada vista puede proyectarse según los datos que necesita, sin imponer una única estructura a consultas con requisitos distintos.

6.2 Proyecciones y consistencia eventual

Una proyección transforma cambios del lado de escritura en una vista optimizada para consultas. Si la actualización de la proyección ocurre asincrónicamente, puede existir un intervalo breve en el que el comando ya tuvo éxito, pero la lectura aún devuelve el estado anterior. Esa propiedad se llama **consistencia eventual**.

La interfaz y el contrato deben tomar en cuenta ese intervalo. Si el cliente acaba de confirmar una orden, puede recibir el nuevo estado en la respuesta del comando o consultar una vista con una garantía explícita de actualización. Si presenta una vista atrasada como si fuera actual, una persona podría repetir una operación o interpretar mal su estado.

La separación lógica no exige mensajería. Si se usan eventos o colas para actualizar otro almacén, aparecen responsabilidades adicionales: fallos de publicación, duplicados, reintentos, orden, idempotencia y observación de proyecciones. El patrón CQRS puede ser tan simple como dos modelos de código con una base compartida.

En muchos comercios, las lecturas superan a las escrituras. Separar responsabilidades permite optimizar ambos lados; el escalado independiente requiere además separar almacenes o desplegar componentes de lectura y escritura por separado. El modelo de lectura puede aplicar técnicas de caché o réplicas de solo lectura para vistas que toleran datos atrasados; los comandos deben volver a validar las invariantes en el modelo de escritura.

CQRS ayuda cuando las lecturas y escrituras tienen necesidades significativamente diferentes: forma de datos, complejidad, rendimiento, permisos o volumen. Si una misma estructura y un conjunto de consultas CRUD satisfacen el problema, CQRS puede duplicar modelos, conversiones y diagnóstico sin un beneficio suficiente.

6.3 Event Sourcing: persistir los hechos, no solo el estado actual

CQRS y Event Sourcing son decisiones distintas. CQRS separa las responsabilidades o modelos de lectura y escritura; Event Sourcing conserva los cambios como una secuencia de eventos. Pueden adoptarse por separado o combinarse. Al combinarlos, el event store es la fuente de verdad del lado de escritura y sus eventos pueden alimentar proyecciones, sin exigir que lectura y escritura usen almacenes separados.

En CRUD tradicional se suele guardar el estado actual: por ejemplo, `Orden.estado = "Confirmada"`. Una actualización reemplaza el valor anterior; si no existe otro registro, no se sabe qué secuencia de decisiones

produjo ese estado.

En **Event Sourcing**, el sistema conserva una secuencia durable de eventos que describe cambios significativos de un agregado. El estado actual se reconstruye aplicando los eventos en orden. El event store conserva esa secuencia como fuente de verdad del agregado.



El evento describe algo que ocurrió, como `OrdenCreada`, `PagoConfirmado` o `OrdenCancelada`. No describe únicamente cómo quedó una columna, como `EstadoActualizado`. Un comando expresa algo que se quiere hacer; puede rechazarse y, en ese caso, no produce el evento esperado.

Un evento de dominio puede ser parte del modelo que reacciona a un hecho. En un diseño Event Sourcing, los eventos persistidos del agregado forman el historial canónico de sus transiciones. Eso no convierte automáticamente cada evento almacenado en un mensaje público para otros sistemas. Para integrar contextos pueden publicarse contratos externos distintos, con su propia evolución y garantías.

6.4 Flujo de escritura y concurrencia

Un flujo de escritura común es:

1. Cargar la secuencia del agregado, o un snapshot más los eventos posteriores.
2. Reconstruir el estado actual aplicando los eventos en orden.
3. Enviar el comando al agregado y verificar las invariantes.
4. Si el comando es válido, generar uno o más eventos que expresen los hechos producidos.
5. Anexar los eventos al stream con la versión esperada y confirmar el resultado.

La versión esperada implementa **control optimista de concurrencia**. Si dos operaciones cargan la misma versión y ambas intentan cambiar la misma orden, solo puede aceptarse una escritura sobre esa versión. La otra se rechaza por conflicto; el caso de uso recarga el estado actualizado y vuelve a evaluar el comando, sin repetirlo a ciegas porque la regla de negocio podría ya no permitirlo. Event Sourcing no hace desaparecer la contención: el agregado y su stream deben seguir abarcando el límite de consistencia que se quiere proteger.

6.5 Lecturas, snapshots y evolución

Para consultar un stream corto, el sistema puede reproducir sus eventos. Si crece mucho, puede guardar un **snapshot** con el estado reconstruido en una versión determinada y luego reproducir solo los eventos posteriores. El snapshot reduce trabajo de reconstrucción; no sustituye el stream, que sigue siendo la fuente de verdad.

Las consultas sobre muchos agregados suelen servirse mediante **proyecciones** o vistas materializadas. Un procesador lee eventos y actualiza un modelo de lectura adaptado a consultas como “historial reciente de esta orden”. Las proyecciones pueden reconstruirse o rediseñarse leyendo eventos anteriores, aunque hacerlo

puede tomar tiempo y recursos. Si se actualizan de forma asíncrona, el modelo de lectura es eventualmente consistente.

Los eventos almacenados son contratos duraderos: al cambiar su significado, el código nuevo debe seguir interpretando eventos anteriores, con versiones o transformaciones durante la lectura. No se reescriben como si fueran filas actuales — un error de negocio se corrige con un hecho compensatorio que conserva lo ocurrido. El historial registra los hechos de negocio que el modelo decidió guardar, no una auditoría técnica automática de cada acción; el diseño debe elegir eventos con significado, sin volcar indiscriminadamente todo lo que el sistema recibe o calcula.

6.6 Event store y mensajería

Un **event store** no es sinónimo de un **message broker**: resuelven necesidades distintas.

	Event store	Message broker
Rol	Fuente de verdad del agregado	Distribuye mensajes entre servicios
Lectura	Por stream de un agregado, con su secuencia ordenada	Por tópico, partición y posición del consumidor; el orden depende de las garantías del broker y su particionado
Concurrencia	Puede comprobar la versión esperada del stream al anexas eventos	No ofrece por sí solo el control de versión esperada del agregado; debe resolverse en la aplicación o con otra garantía
Retención	Conserva la historia necesaria para reconstruir el estado, sujeta a políticas explícitas	Es configurable; puede retener o compactar registros, pero eso no la convierte por sí solo en la fuente de verdad de cada agregado

Kafka puede distribuir eventos hacia proyecciones, pero una cola o un log de mensajes no sustituye por sí solo las garantías de un event store.

6.7 Cuándo puede convenir y cuándo no

Puede justificarse cuando el producto necesita consultar cómo evolucionó una entidad, auditar decisiones de negocio, reconstruir estados o crear distintas vistas a partir del mismo historial. El valor debe superar los costos de versionar eventos, resolver conflictos, procesar proyecciones, mantener el event store y responder consultas.

Para un CRUD con reglas sencillas y sin necesidad de historial, Event Sourcing añade un modo de persistencia más difícil de cambiar. También resulta poco atractivo para datos de referencia casi inmutables. Si el producto requiere que las lecturas reflejen de inmediato un cambio y las proyecciones se actualizan de forma asíncrona, el equipo debe aceptar la demora o diseñar una actualización sincrónica; Event Sourcing por sí mismo no impone lecturas asíncronas. La estrategia puede aplicarse a una parte del sistema —por ejemplo, el ciclo de vida de una orden— mientras perfiles y preferencias siguen usando persistencia de estado actual.

7. Arquitectura reactiva y programación reactiva

7.1 Dos niveles que comparten vocabulario

El término **reactivo** se usa para ideas relacionadas pero distintas:

- Un **Reactive System** es un sistema que busca responder oportunamente, adaptarse a cambios de carga, contener fallos y comunicarse mediante paso asíncrono de mensajes.
- La **programación reactiva** es un modelo de programación para componer resultados asíncronos y flujos de datos, por ejemplo procesar cada elemento a medida que llega.

Un sistema puede tener componentes con programación reactiva sin cumplir por ello todas las propiedades de un Reactive System. Del mismo modo, una arquitectura distribuida puede intercambiar mensajes asíncronos sin que su código use una biblioteca de streams reactivos.

7.2 I/O no bloqueante, streams y backpressure

En un modelo con I/O bloqueante, el hilo que procesa una solicitud puede quedar esperando mientras responde una base de datos o servicio remoto. Con I/O no bloqueante, el hilo puede atender otro trabajo mientras la operación espera; cuando llega el resultado, continúa desde el punto correspondiente. Esto permite atender muchas esperas concurrentes con menos hilos, siempre que el stack completo soporte el modelo.

En un modelo de streams reactivos se habla de **Publisher** como productor y **Subscriber** como consumidor. Un flujo puede emitir cero, uno o muchos resultados a lo largo del tiempo. Las operaciones de transformación y composición expresan qué hacer cuando cada resultado llega y cómo propagar errores o cancelaciones.

Backpressure es la coordinación de demanda entre consumidor y productor. Si el consumidor procesa más lentamente que el productor, el flujo debe regular la producción, rechazar, almacenar dentro de un límite o aplicar otra política explícita. Sin esa coordinación, un flujo rápido puede acumular trabajo en memoria hasta saturar el servicio.

Atender más I/O en paralelo no significa ejecutar más trabajo de CPU al mismo tiempo ni hacer más rápida una dependencia lenta. Las ventajas del modelo se reducen si una operación bloqueante ocupa el hilo de I/O, si la base no ofrece un cliente no bloqueante o si el trabajo de CPU monopoliza los ejecutores compartidos. La cadena completa debe sostener las mismas expectativas de ejecución y límites.

7.3 Trade-offs

El modelo reactivo puede ser útil con muchas esperas de I/O concurrentes, flujos continuos y necesidad de ajustar demanda. También introduce una forma diferente de razonar sobre composición, fallos, cancelación, propagación de contexto, stack traces y pruebas. Cuando la aplicación tiene poca concurrencia, carga principalmente de CPU o un flujo de control simple, el modelo imperativo puede ser más claro y suficiente.

Los conceptos de resiliencia vistos anteriormente siguen siendo necesarios. Un stream no elimina timeouts, circuit breakers o límites de concurrencia. La programación reactiva dentro de un proceso tampoco equivale a una cola durable ni define por sí misma cómo se comunican servicios distribuidos.

En Go, el runtime usa un *netpoller* para operaciones de I/O en descriptores compatibles, como muchas conexiones de red: una goroutine que espera se suspende y el hilo del sistema operativo puede ejecutar otra goroutine. Un Handler de Gin que espera un driver de base de datos apoyado en ese camino puede conservar control de flujo imperativo sin retener necesariamente un hilo durante toda la espera. La goroutine, la solicitud y sus conexiones siguen ocupando recursos, y no toda llamada al sistema o driver usa el *netpoller*.

Bibliotecas como Reactor ofrecen una forma de componer operaciones y flujos asíncronos, con manejo de demanda y backpressure donde aplica el protocolo correspondiente. Los *virtual threads* de Java permiten otra alternativa para atender muchas operaciones de I/O con código imperativo; no reemplazan las capacidades de composición de flujos ni el control de demanda de Reactive Streams. Cada modelo tiene costos distintos de composición, depuración y uso de recursos.

8. Frameworks y bibliotecas como referencias

Los frameworks y bibliotecas ofrecen contratos, adaptadores, buses, tipos asíncronos o mecanismos de verificación. Reducen trabajo de implementación, pero no sustituyen el razonamiento sobre reglas, límites y costos.

Herramienta	Qué facilita	Qué sigue siendo decisión del equipo
Gin (Go)	Recibir requests HTTP y generar respuestas.	Mantener el Handler como adaptador, definir casos de uso y decidir las dependencias internas.
Watermill (Go)	Abstracciones de mensajería y building blocks de CQRS sobre Pub/Sub y routers.	Definir los contratos del dominio y separar modelos; no provee por sí solo un event store ni decide si hace falta CQRS.
Fx (Go)	Inyección de dependencias y ciclo de vida de una aplicación; ensambla componentes al iniciar.	Diseñar qué depende de qué; la herramienta ensambla el grafo, no decide la dirección de las dependencias.
Wire (Go, archivado)	Referencia histórica de generación de código para inyección de dependencias en tiempo de compilación.	El repositorio fue archivado en 2025; no presentarlo como opción activa para proyectos nuevos.
Axon Framework (JVM)	Building blocks para DDD, CQRS y Event Sourcing, como comandos, eventos, consultas y event stores.	Definir límites, modelo de dominio, eventos relevantes y si la complejidad justifica Event Sourcing.
Spring Modulith (JVM)	Organizar y verificar módulos de una aplicación Spring Boot con orientación de dominio.	Definir módulos y sus relaciones; un modulith no separa automáticamente despliegues.
jMolecules (Java)	Expresar en el código conceptos de DDD y estilos como Hexagonal, y conectarlos con reglas de verificación.	Crear límites válidos y corregir dependencias que rompen las reglas.
Spring WebFlux / Reactor (JVM)	Stack web no bloqueante y soporte de Reactive Streams con <code>Mono</code> / <code>Flux</code> .	Evitar I/O bloqueante y establecer la política de concurrencia, errores y demanda.
Quarkus / Mutiny (JVM)	APIs reactivas para componer operaciones asíncronas y flujos; permite combinar código reactivo e imperativo.	Aplicar el modelo en las partes que lo justifican y sostenerlo en todas las dependencias necesarias.

Watermill y Fx muestran que algunas de estas decisiones también se facilitan en Go; Wire queda como antecedente de inyección de dependencias por generación de código. El resto de los ejemplos pertenece al ecosistema JVM porque varios frameworks especializados allí documentan estas capacidades de forma más explícita. La arquitectura no exige adoptar ninguna de estas herramientas: lo que facilitan puede reconocerse y resolverse a mano en cualquier plataforma.

9. Casos reales

9.1 Netflix Studio Workflows y Hexagonal

Netflix describió la construcción de aplicaciones internas para los flujos de producción de Studio. Una aplicación nueva necesitaba datos distribuidos entre múltiples sistemas y protocolos, mientras se anticipaba que algunas fuentes migrarían desde un monolito hacia servicios especializados.

El equipo adoptó principios de Hexagonal para aislar las reglas de negocio de las fuentes. Cuando necesitó cambiar una lectura desde una API JSON hacia una fuente GraphQL, implementó otro adaptador para el contrato existente. Netflix reportó que ese cambio tomó dos horas ([Netflix Technology Blog](#)). Es un resultado situado en un cambio concreto y muestra cómo separar la lógica de negocio del formato y ubicación de los datos puede reducir su alcance.

9.2 Uber DOMA y el ordenamiento por dominio

Uber presentó **Domain-Oriented Microservice Architecture (DOMA)** para agrupar servicios relacionados en dominios con gateways e interfaces definidos. El artículo vincula su enfoque con principios de DDD, Clean Architecture, diseño orientado a servicios y diseño basado en interfaces.

DOMA muestra que, a escala organizacional, no basta con tener servicios pequeños: también se necesita que las dependencias y ownership formen una estructura comprensible. La experiencia de Uber no implica que una plataforma estudiantil necesite agrupar servicios del mismo modo; aporta un ejemplo de cómo los límites de dominio pueden ayudar a reducir complejidad entre muchos equipos y servicios ([Uber Engineering](#)).

10. Diseño de referencia: el interior de Órdenes

La plataforma necesita confirmar órdenes, consultar el historial de un cliente e incorporar recomendaciones generadas por un proveedor de IA. Se comparan alternativas sin fijar de antemano una única combinación.

10.1 Modelar responsabilidades y límites

Órdenes controla reglas de su ciclo de vida: por ejemplo, una orden no se cancela después de despacharse. Pagos conserva la semántica de autorización y devolución. Recomendaciones conoce cómo producir sugerencias. Estos módulos o contextos pueden vivir juntos o separados según otras necesidades de despliegue.

Dentro de Órdenes, `Orden` puede ser un agregado con identidad y reglas de transición. `Monto` y `DirecciónDeEntrega` pueden ser objetos de valor si sus atributos determinan igualdad y se reemplazan como unidades. La raíz del agregado valida cada cambio aceptado; la base no debe permitir transiciones inválidas solo porque una columna admite cualquier string.

10.2 Aislar detalles con puertos y adaptadores

El caso de uso `ConfirmarOrden` se invoca mediante un puerto de entrada. Para guardar el agregado utiliza un puerto de salida de repositorio. El Handler HTTP, el repositorio SQL y cualquier cliente de IA son

adaptadores. El núcleo no importa las estructuras del framework, del ORM o del SDK.

Para generar sugerencias, el sistema define una operación propia —por ejemplo, recomendar productos relevantes a partir de una lista de identificadores— y la separa del contrato específico de cada proveedor. Si se reemplaza el modelo de IA, el adaptador nuevo traduce solicitudes y respuestas. Las reglas de Órdenes siguen decidiendo qué ocurre si la recomendación no está disponible o no resulta válida.

Clean Architecture puede describir el mismo límite mediante las dependencias hacia adentro. Ninguno de los dos modelos exige mover cada módulo a un servicio separado ni agregar abstracciones que no protegen cambios reales.

10.3 Decidir si separar lecturas y escrituras

Una operación de escritura valida y cambia la orden. Una consulta de historial devuelve una vista compacta de órdenes anteriores. Si esas formas, reglas o cargas difieren, CQRS permite mantener modelos separados. Al principio pueden compartir base y transacción; separar almacenes agrega sincronización y posible consistencia eventual.

Si la consulta devuelve exactamente los mismos datos que el modelo de escritura y la carga no necesita optimización independiente, un repositorio CRUD compartido puede ser más simple y fácil de mantener.

10.4 Evaluar Event Sourcing para el ciclo de vida

Si el producto necesita reconstruir las decisiones del ciclo de vida de una orden, el agregado puede persistir hechos como `OrdenCreada`, `PagoConfirmado` y `OrdenCancelada`. El event store es la fuente de verdad del estado del agregado. Una vista de historial puede proyectarse desde esos eventos para lectura.

Cada append verifica la versión esperada del stream. Si una operación concurrente avanzó la versión, el caso de uso recarga el estado y vuelve a validar; el mismo comando puede ya no ser válido. La vista proyectada puede actualizarse después del append, de modo que la aplicación debe tener en cuenta una demora temporal.

Esta decisión agrega beneficios potenciales de historia y reconstrucción, y costos de evolución, proyecciones y operación. Si el requisito es solo mostrar el estado actual de una orden, persistir su estado directamente puede bastar.

10.5 Evaluar el modelo reactivo

Si recomendaciones recibe muchas solicitudes simultáneas y el acceso externo es I/O-bound, un cliente no bloqueante puede liberar hilos mientras espera. La ganancia requiere que las demás etapas de acceso y procesamiento respeten el modelo y que la demanda se limite.

Una llamada reactiva no vuelve más rápida una inferencia lenta ni debe bloquear recursos centrales de Órdenes. Si el volumen y la concurrencia son moderados, una integración imperativa con timeout puede ser más clara. La elección se valida con necesidades y medidas reales; no se deduce del uso de IA.

11. Trade-offs y criterios de decisión

Pregunta	Alternativa que puede ayudar	Costos que deben justificarse
¿Las reglas de negocio son ambiguas o cambian junto con un lenguaje del dominio?	DDD para modelar términos, invariantes y contextos.	Colaboración con expertos, aprendizaje del modelo y mantenimiento de límites explícitos.
¿El núcleo depende de una base, framework o proveedor que cambia?	Hexagonal o Clean Architecture para dirigir dependencias hacia las reglas.	Contratos, mapeos y estructura adicional; interfaces sin variación relevante pueden ser sobreingeniería.
¿Lecturas y escrituras necesitan formas, reglas o cargas distintas?	CQRS para separar responsabilidades y, cuando haga falta, modelos.	Duplicación, proyecciones y consistencia eventual si se desacoplan almacenes.
¿El historial de cambios es valioso para el producto o para reconstruir estado?	Event Sourcing para conservar hechos y derivar estado.	Evolución de eventos, conflictos concurrentes, proyecciones y operación del event store.
¿Hay mucho I/O concurrente y el stack puede ser no bloqueante?	Programación reactiva para componer operaciones y regular demanda.	Complejidad de ejecución, depuración, backpressure y propagación de errores/contexto.

Las opciones pueden complementarse, pero el beneficio de cada una debe permanecer visible. Un modelo de dominio preciso puede estar dentro de puertos y adaptadores; CQRS puede usarse con CRUD; Event Sourcing puede ser la persistencia de una parte específica; una operación reactiva puede convivir con flujos imperativos.

Una buena decisión también explicita qué no se optimiza. Si el equipo no necesita intercambiar proveedores, el puerto genérico puede no aportar valor. Si la lectura y escritura comparten requisitos, CQRS añade pasos. Si el historial no es central, un log append-only puede ser una carga. Si no hay presión de concurrencia I/O, el modelo reactivo puede dificultar la comprensión más de lo que ayuda.

12. Puntos clave

- El estilo arquitectónico responde una pregunta diferente del modelo de dominio, la persistencia, el framework o el despliegue.
- DDD ayuda a expresar lenguaje, reglas y límites; no obliga a usar microservicios ni es requisito para Hexagonal o Clean Architecture.
- Un agregado protege invariantes dentro de su límite de consistencia; no equivale automáticamente a una tabla ni a un microservicio.
- Hexagonal y Clean Architecture mantienen las reglas internas independientes de transportes, bases y frameworks mediante dependencias dirigidas hacia adentro.

- CQRS separa responsabilidades de comando y consulta; puede comenzar con modelos lógicos distintos y un mismo almacén de datos.
- Event Sourcing persiste una secuencia de hechos como fuente de verdad; puede respaldar CQRS, pero ambos patrones son decisiones separadas.
- Event Sourcing requiere gestionar versiones de stream, concurrencia, evolución de eventos y proyecciones; sus eventos no son lo mismo que los mensajes de un broker.
- Programación reactiva ofrece composición no bloqueante y backpressure si la cadena completa coopera; no elimina latencia ni reemplaza resiliencia.
- MVC y CRUD siguen siendo opciones válidas cuando expresan el problema con menos complejidad y el equipo puede mantenerlas con claridad.

13. Referencias

- Eric Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.
- Microsoft Learn, [Use Domain Analysis to Model Microservices](#) y [Use Tactical DDD to Design Microservices](#).
- Alistair Cockburn, [Hexagonal Architecture](#).
- Robert C. Martin, [The Clean Architecture](#).
- Microsoft Learn, [CQRS Pattern](#) y [Event Sourcing Pattern](#).
- Netflix Technology Blog, [Ready for changes with Hexagonal Architecture](#).
- Uber Engineering, [Introducing Domain-Oriented Microservice Architecture](#).
- Axon Framework, [Reference Guide](#); Spring, [Spring Modulith](#); [jMolecules](#).
- Watermill, [CQRS component](#); Uber, [Fx](#); Google, [Wire repository and project status](#).
- Go, [internal/poll package](#); OpenJDK, [JEP 444: Virtual Threads](#).
- Apache Kafka, [Design](#) y [Topic-Level Configs](#).
- Spring Framework, [Spring WebFlux](#) y [Reactive Libraries](#).
- Quarkus, [Reactive Architecture](#) y [Mutiny](#).
- [The Reactive Manifesto](#).